



COURSE CODE/TITLE: CPE 010

SECTION: CPE21S2

DATE PERFORMED: July 29, 2026

DATE SUBMITTED: July 29, 2026

NAME: Johrel Louie T. Silao

INSTRUCTOR: Engr. Jimlord M. Quejado

ACTIVITY NO. 4.1 Array-Based vs. STL Stack Implementations

1. PROCEDURE OUTPUT

ILO A: Implement a stack using the C++ STL (`std::stack`).

```
1  #include <iostream>
2  #include <stack>
3
4  int main () {
5
6      // Testing ground of Stack
7      std::stack<int> stack1;
8      std::stack<int> stack2;
9      std::cout<<"=====Testing the Stack STL===== " << std::endl;
10
11     // isEmpty
12     std::cout<<"Is the stack empty? " << stack1.empty() << std::endl;
13
14     // Push
15     std::cout << "=====Push Stack===== " << std::endl;
16     stack1.push(10);
17     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
18     stack1.push(9);
19     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
20     stack1.push(8);
21     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
22     stack1.push(7);
23     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
24     stack1.push(6);
25     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
26     stack1.push(5);
27     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
28     std::cout << "The size of the stack is: " << stack1.size() << std::endl;
29     std::cout << std::endl;
30
31     // Pop
32     stack1.pop();
33     std::cout << "=====Pop Stack===== " << std::endl;
34     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
35     std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
36     std::cout << "The size of the stack is: " << stack1.size() << std::endl;
37
38     // Eemplace
39     std::cout << "=====Emplace Stack===== " << std::endl;
40     stack1.emplace(1); // Replaces the top value of the stack
41     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
42     std::cout << "The size of the stack is: " << stack1.size() << std::endl;
43
44     // Swap
45     stack2.push(4);
46     stack2.push(3);
47     stack2.push(2);
48     stack1.swap(stack2); // Swaps the contents of stack1 and stack2
49
50     //Swapped contents of stack 1 and stack 2
51     std::cout << "=====Swap Stack===== " << std::endl;
52     std::cout << "The top of the stack 1 is: " << stack1.top() << std::endl;
53     std::cout << "The top of the stack 2 is: " << stack2.top() << std::endl;
54
55     std::cout << "The size of the stack 1 is: " << stack1.size() << std::endl;
56     std::cout << "The size of the stack 2 is: " << stack2.size() << std::endl;
57     return 0;
58 }
```

Output:

```
=====Testing the Stack STL=====
Is the stack empty? 1
=====Push Stack=====
The top of the stack is: 10
The top of the stack is: 9
The top of the stack is: 8
The top of the stack is: 7
The top of the stack is: 6
The top of the stack is: 5
The size of the stack is: 6

=====Pop Stack=====
The top of the stack is: 6
Is the stack empty? 0
The size of the stack is: 5
=====Emplace Stack=====
The top of the stack is: 1
The size of the stack is: 6
=====Swap Stack=====
The top of the stack 1 is: 2
The top of the stack 2 is: 1
The size of the stack 1 is: 3
The size of the stack 2 is: 6
```

ILO B.1: Implement a stack using a Static Array.

```

1  #include <iostream>
2
3      // Global Declarations
4  #define maxCap 10
5      int choice;
6      int stackArr [maxCap];
7      int top = -1, newData;
8
9      // Prototype Functions
10     void push();
11     void pop();
12     void Top();
13     bool isEmpty();
14     bool isFull();
15     void displayStack();
16
17 int main() {
18
19     // Main Driver
20     while (true) {
21         std::cout << "====Stack Operations:====" << std::endl;
22         std::cout << "1. Push Stack" << std::endl;
23         std::cout << "2. Pop Stack" << std::endl;
24         std::cout << "3. Top Stack" << std::endl;
25         std::cout << "4. isEmpty Stack" << std::endl;
26         std::cout << "5. isFull Stack" << std::endl;
27         std::cout << "6. Display the stack" << std::endl;
28         std::cout << "6. Exit the program" << std::endl;
29         std::cout << "====" << std::endl;
30         std::cin >> choice;
31
32         if (choice == 6) {
33             std::cout << "Exiting the Program!" << std::endl;
34             break;
35         }
36
37         switch (choice) {
38             case 1: push();
39                 break;
40             case 2: pop();
41                 break;
42             case 3: Top();
43                 break;
44             case 4: std::cout << "Is the stack empty? " << isEmpty() << std::endl;
45                 break;
46             case 5: std::cout << "Is the stack full? " << isFull() << std::endl;
47                 break;
48             case 6: displayStack();
49                 break;
50             default: std::cout << "Invalid Choice" << std::endl;
51         }
52     }
53
54     return 0;
55 }

```

```

57 // Function Definition
58
59 bool isEmpty() {
60     // Error checking
61     if (top == -1) return true;
62     return false;
63 }
64
65 bool isFull() {
66     if (top == maxCap - 1) return true;
67     return false;
68 }
69
70 void push() {
71     // Error checking
72     if (isFull()) {
73         std::cout << "Stack Overflow" << std::endl;
74         return;
75     }
76     // Pushing to the stack
77     std::cout << "Enter a new value: " << std::endl;
78     std::cin >> newData;
79     stackArr[++top] = newData;
80 }
81
82 void pop() {
83     // Error checking
84     if (isEmpty()) {
85         std::cout << "Stack Underflow" << std::endl;
86         return;
87     }
88
89     // Displays the "popped" value of the stack
90     std::cout << "Value Popped: " << stackArr[top] << std::endl;
91
92     // Decrement the top value from the stack
93     top--;
94 }
95
96 void Top () {
97     // Error Catching
98     if (isEmpty()) {
99         std::cout << "The stack is empty!" << std::endl;
100         return;
101     }
102
103     // Checks the top value:
104     std::cout << "Top Element: " << stackArr[top] << std::endl;
105 }

```

```

100
107 //Displays the elements of the stack
108 void displayStack() {
109     if (isEmpty()) {
110         std::cout << "The stack is empty!" << std::endl;
111     }
112     for (int i = top; i >= 0; i--) {
113         std::cout << stackArr[i] << ",";
114     }
115     std::cout << std::endl;
116 }

```

Output:

### Stack Operation Output: Push Stack

```

=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Enter a new value:
1

```

```

=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Enter a new value:
2

```

```

=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Enter a new value:
3

```

```

=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Enter a new value:
4

```

=====Stack Operations:=====

1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program

=====

1  
Enter a new value:  
5

=====Stack Operations:=====

1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program

=====

1  
Enter a new value:  
6

=====Stack Operations:=====

1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program

=====

1  
Enter a new value:  
7

=====Stack Operations:=====

1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program

=====

1  
Enter a new value:  
8

=====Stack Operations:=====

1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program

=====

1  
Enter a new value:  
9

=====Stack Operations:=====

1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program

=====

1  
Enter a new value:  
10

=====Stack Operations:=====

1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program

=====

1  
Stack Overflow

### Stack Operation Output: Pop Stack

```
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
2
Value Popped: 10
```

### Stack Operation Output: Top Stack

```
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
3
Top Element: 9
```

### Stack Operation Output: isEmpty Stack

```
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
4
Is the stack empty? 0
```

### Stack Operation Output: isFull Stack

```
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
5
Is the stack full? 0
```

### Stack Operation Output: Display the Stack

```
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
7. Exit the program
=====
6
9,8,7,6,5,4,3,2,1,
```

### Stack Operation Output: Exit the Program

```
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
7. Exit the program
=====
7
Exiting the Program!
PS C:\Users\TIPQC>
```

### Conclusion:

In this activity, I learned how the “Stack” implementation works, it works by Last-In, First-Out (LIFO) data structures where elements are added and removed strictly from the top. To prevent errors, I used isEmpty() to return true when the top index is -1, which shows that there are no elements left to access or remove, while isFull() returns true when the top index reaches the max cap size, to signal that there are no more elements to store. The push() stores a new value, while the pop() decrements or removes the value. To check the recent value that is added, the top() is used. Finally the displayStack() checks if the stack contains elements.



## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** \_\_\_\_\_

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** \_\_\_\_\_

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** \_\_\_\_\_

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.